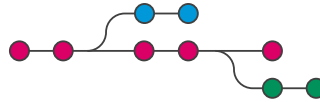




# Systemdesign Version Control



Studiengang Systemtechnik

Silvan Zahno [silvan.zahno@hevs.ch](mailto:silvan.zahno@hevs.ch)



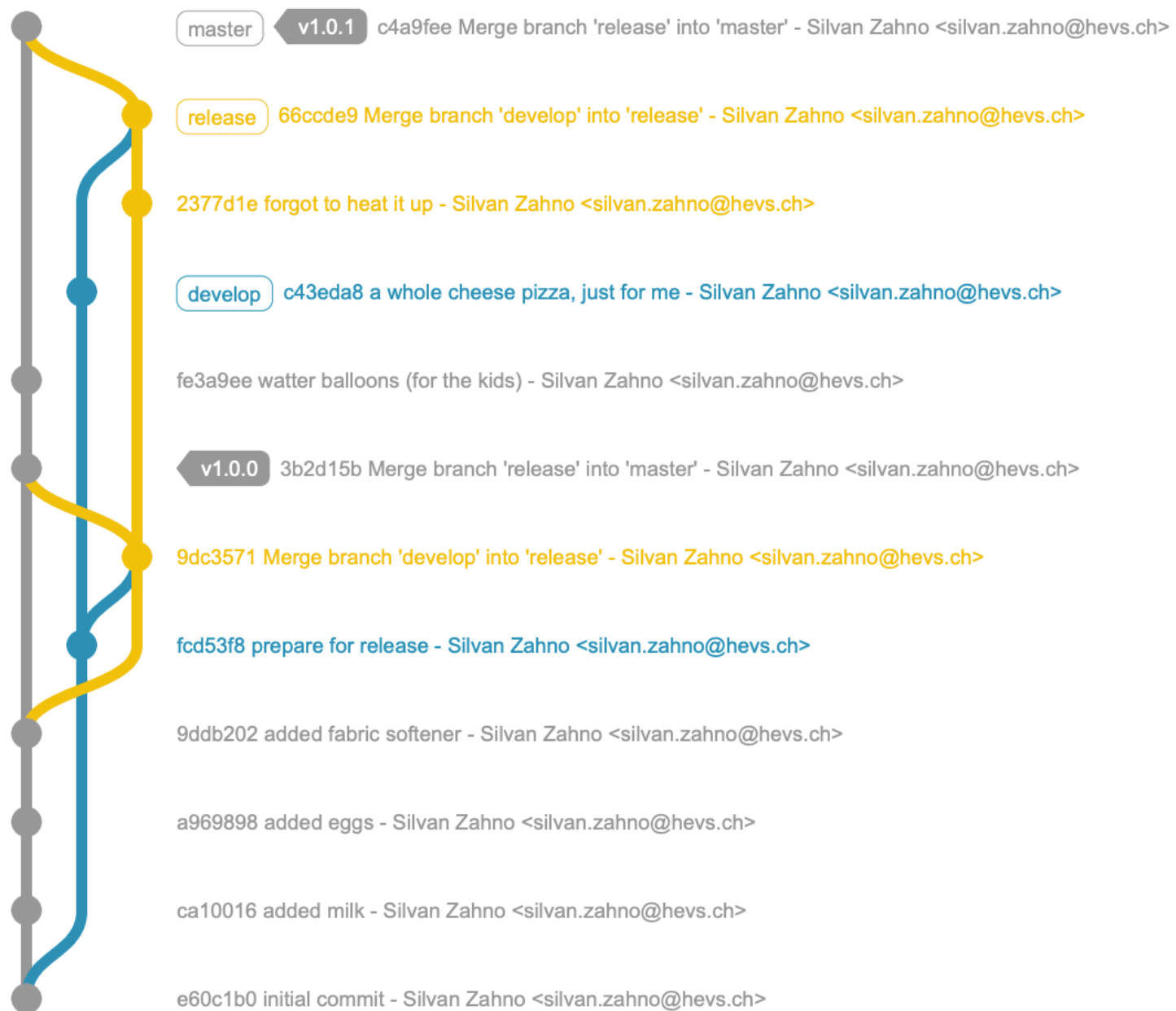
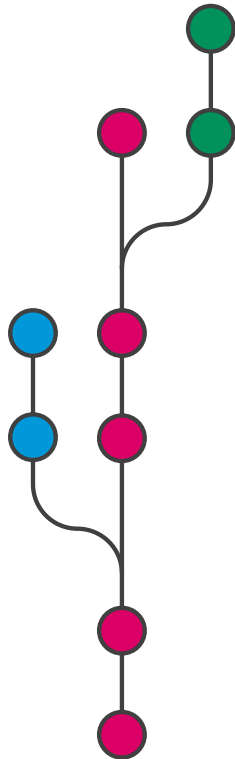
# Ihr derzeitiges System



```
.
├── wichtige-datei copy.txt
├── wichtige-datei v1.txt
├── wichtige-datei v2.1 backup.txt
├── wichtige-datei v2.1 backup.txt.old
├── wichtige-datei v2.1.txt
├── wichtige-datei v2.txt
└── wichtige-datei.txt
```

1 Ordner, 7 Dateien

## ZaS



# Possible Hilfsmittel



Git (git)



Subversion (svn)

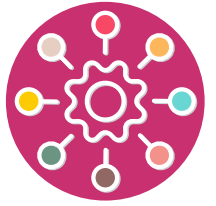
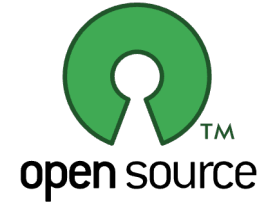


Mercurial (hg)

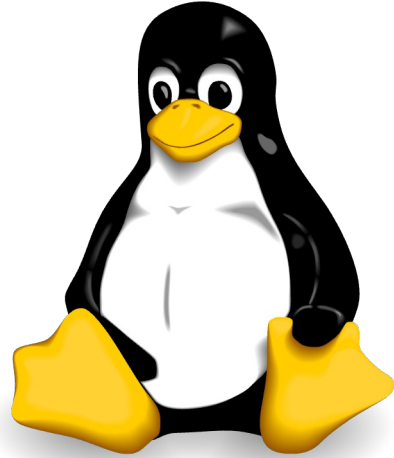




# Linux Torvalds



## Linux (1991)



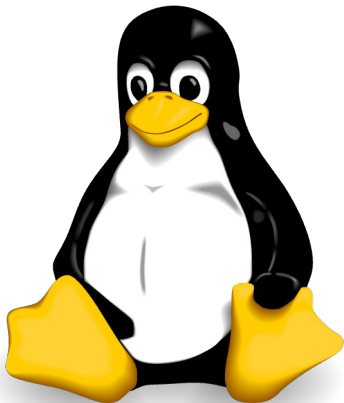
## Git (2005)



# Warum Linux braucht git



Linux hat sich in den letzten 30 Jahren zum grössten gemeinschaftlichen Entwicklungsprojekt in der Geschichte der Computertechnik entwickelt.



- 600 aktive Linux-Distributionen
- 85% aller Smartphones
- 500 Top-Supercomputer
- > 27,8 Millionen Codezeilen
- > 12'000 Mitwirkende
- > 1 Million commits

<https://truelist.co/blog/linux-statistics/>

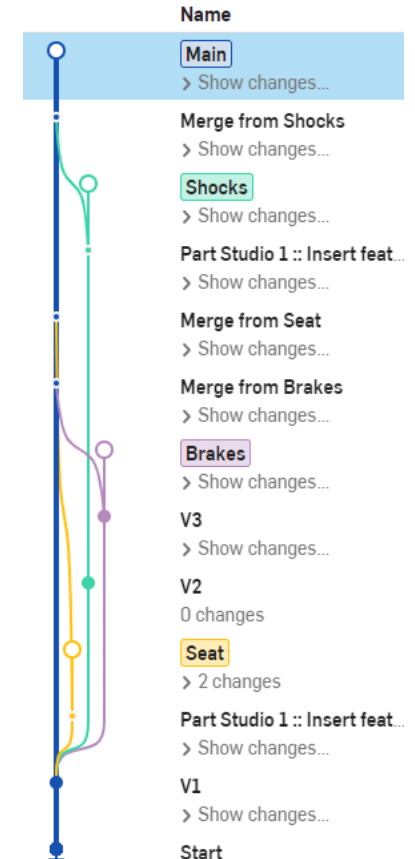
# Warum für ein Power&Control oder Design&Material git notwendig ist

## Versionierung

- Verfolgung von Änderungen
- Zusammenarbeit
- Rollback-Fähigkeiten
- Dokumentation
- Bereitstellung



**AUTODESK**  
**Vault**



# Git Plattformen



## Gitlab

<https://gitlab.com>

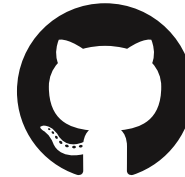
<https://gitlab.hevs.ch>



GitLab

## Github

<https://github.com>



GitHub

## Bitbucket

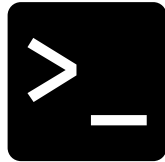
<https://bitbucket.com>



Bitbucket

# Git Hilfsmittel

Kommandozeile



Sublime Merge



Git Cola



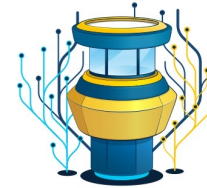
Git Kraken



Fork



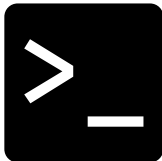
Tower



SmartGit



# Git Kommandozeile



git status

git diff

git add <file>

git diff --staged

git reset <file>

git commit -m "<commit message>"

git fetch <remote>

git merge <remote> <branch>

git push <remote> <branch>

git pull

```
zas@zas@zac ~ - zsh - 99x52
Last login: Tue Mar  8 09:26:26 on ttys004
(zas@zac)~(base)
$ git --help
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
      [--exec-path=<path>] [--html-path] [--man-path] [--info-path]
      [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
      [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
      [--super-prefix=<path>] [--config-env=<name>=<envvar>]
      <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

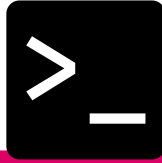
grow, mark and tweak your common history
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

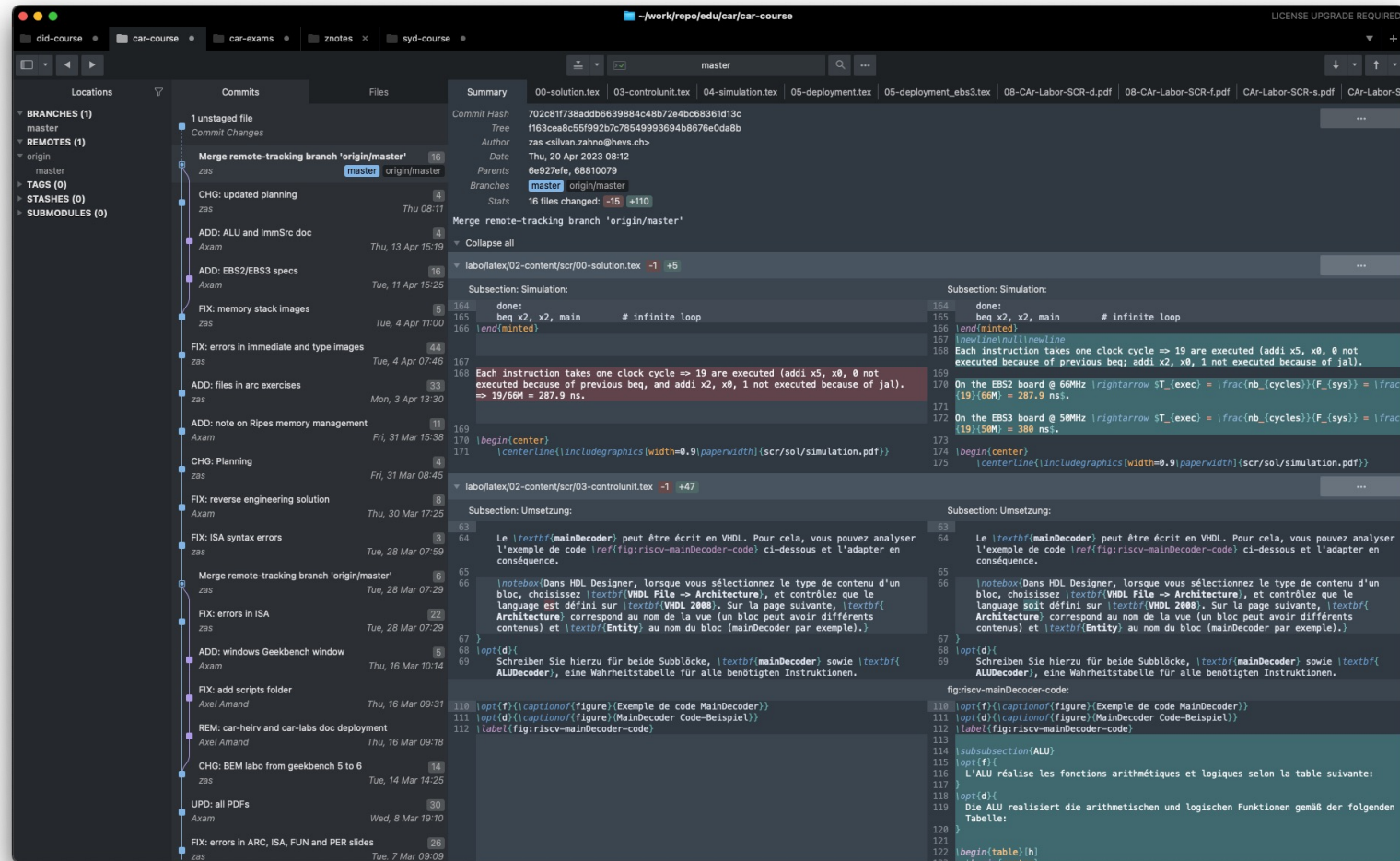
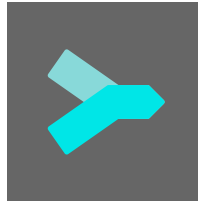
(zas@zac)~(base)
$
```

# git Befehle



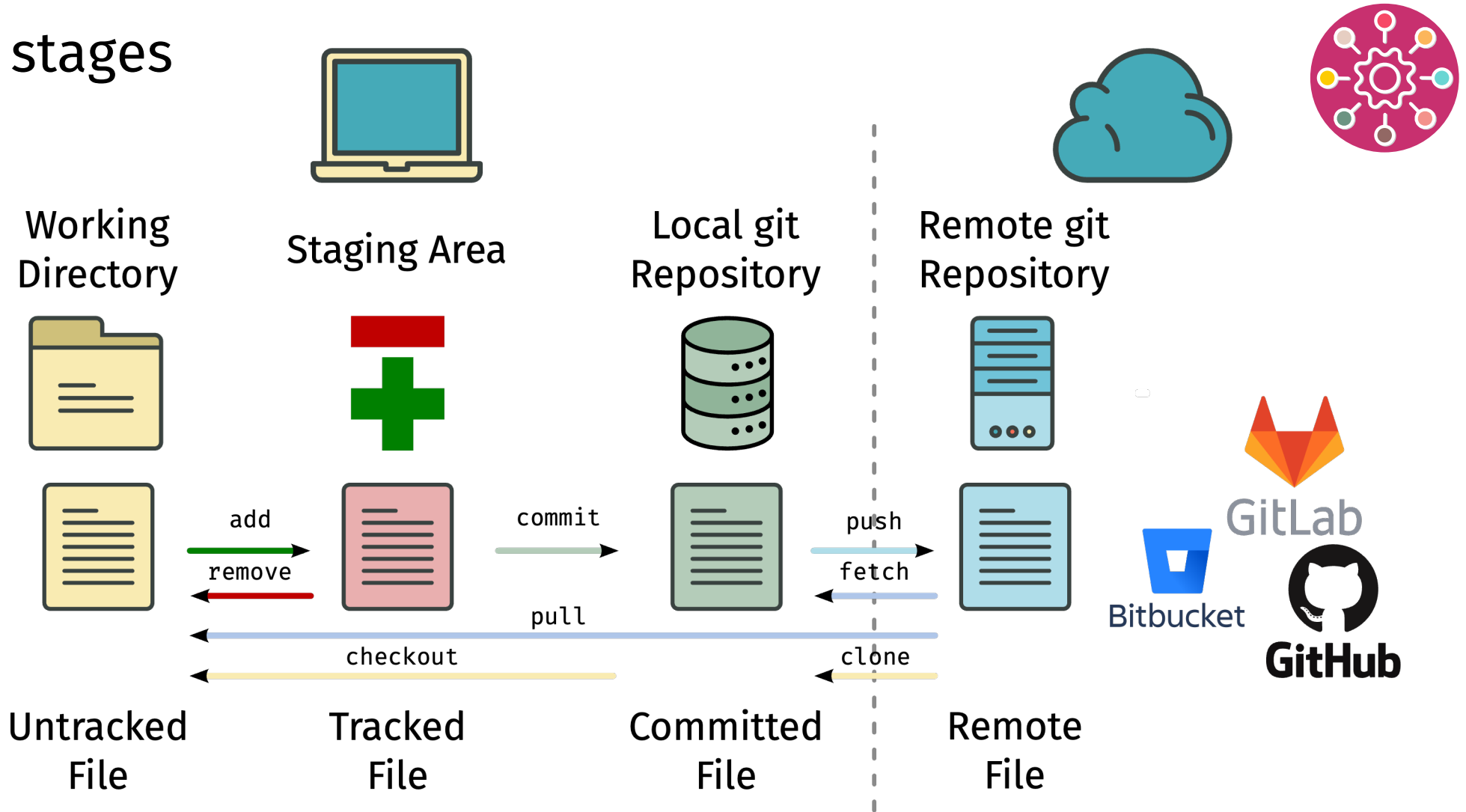
| Befehl                                  |  | Beschreibung                                                                      | Befehl                                                         |  | Beschreibung                                                                                 |
|-----------------------------------------|--|-----------------------------------------------------------------------------------|----------------------------------------------------------------|--|----------------------------------------------------------------------------------------------|
| Eine working area beginnen              |  |                                                                                   | Wachsen, markieren und optimieren Sie Ihre gemeinsame Historie |  |                                                                                              |
| clone                                   |  | Klonen eines Repositorys in ein neues Verzeichnis                                 | branch                                                         |  | Zweige auflisten, erstellen oder löschen<br>Auschecken                                       |
| init                                    |  | Erstellen eines leeren git-Repo oder Reinitialisieren eines vorhandenen Repo      | checkout                                                       |  | Zweige wechseln oder Arbeitsbaumdateien wiederherstellen                                     |
| An aktuellen Änderungen arbeiten        |  |                                                                                   | commit                                                         |  | Änderungen an der Projektliste aufzeichnen                                                   |
| add                                     |  | Hinzufügen von Dateiinhalten zum Index                                            | diff                                                           |  | Änderungen zwischen Commits, Commit und Arbeitsbaum anzeigen, etc.                           |
| mv                                      |  | Verschieben oder Umbenennen einer Datei, eines Verzeichnisses oder eines Symlinks | merge                                                          |  | Zwei oder mehr Entwicklungsgeschichten zusammenführen                                        |
| reset                                   |  | Zurücksetzen des aktuellen HEAD auf den angegebenen Zustand                       | rebase                                                         |  | Commits auf einen anderen Basistipp anwenden<br>tag                                          |
| rm                                      |  | Entfernen von Dateien aus dem Arbeitsbaum und aus dem Index                       | tag                                                            |  | Ein Tag-Objekt erstellen, auflisten, löschen oder verifizieren                               |
| Untersuchen der Historie und des Status |  |                                                                                   | Zusammenarbeiten                                               |  |                                                                                              |
| log                                     |  | Übergabeprotokolle anzeigen                                                       | fetch                                                          |  | Herunterladen von Objekten und Referenzen aus einem anderen Repo                             |
| show                                    |  | Verschiedene Arten von Objekten anzeigen                                          | pull                                                           |  | Abrufen und Integrieren aus einem anderen Projektarchiv oder einem lokalen Zweig<br>schieben |
| status                                  |  | Show the working tree status                                                      | push                                                           |  | Aktualisieren entfernter Referenzen zusammen mit zugehörigen Objekten                        |

# Sublime Merge



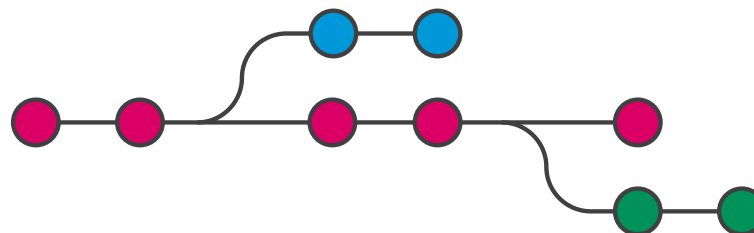


# Git stages





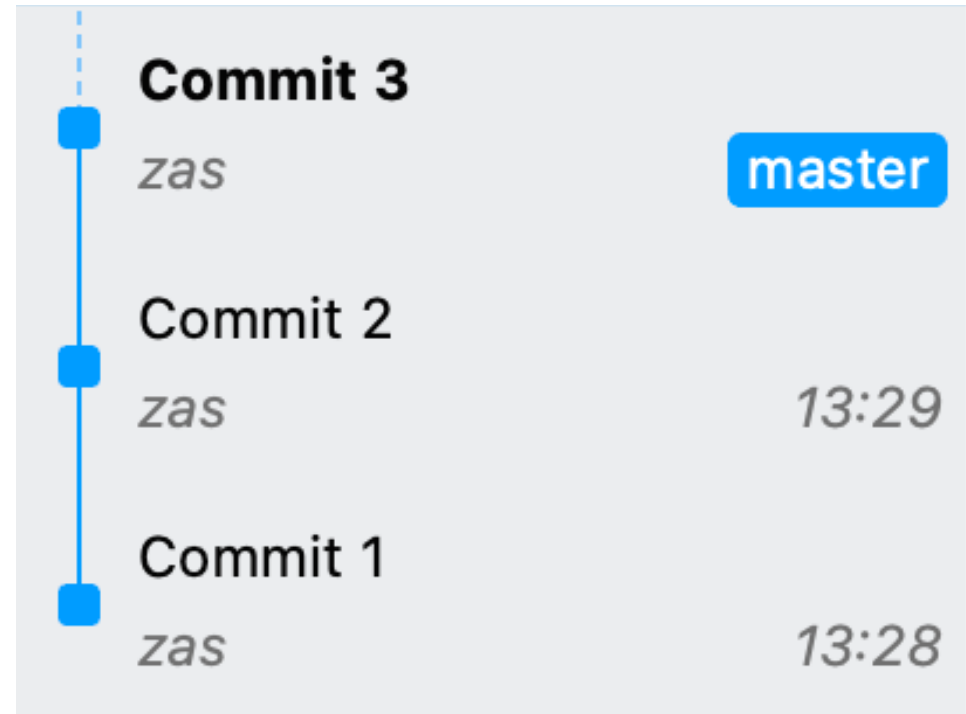
## Git Branch und Merge Beispiele



# Branch und Merge

## Initial repo state

Jedes repo hat entweder ein **main** oder eine **master** branch als standart branch



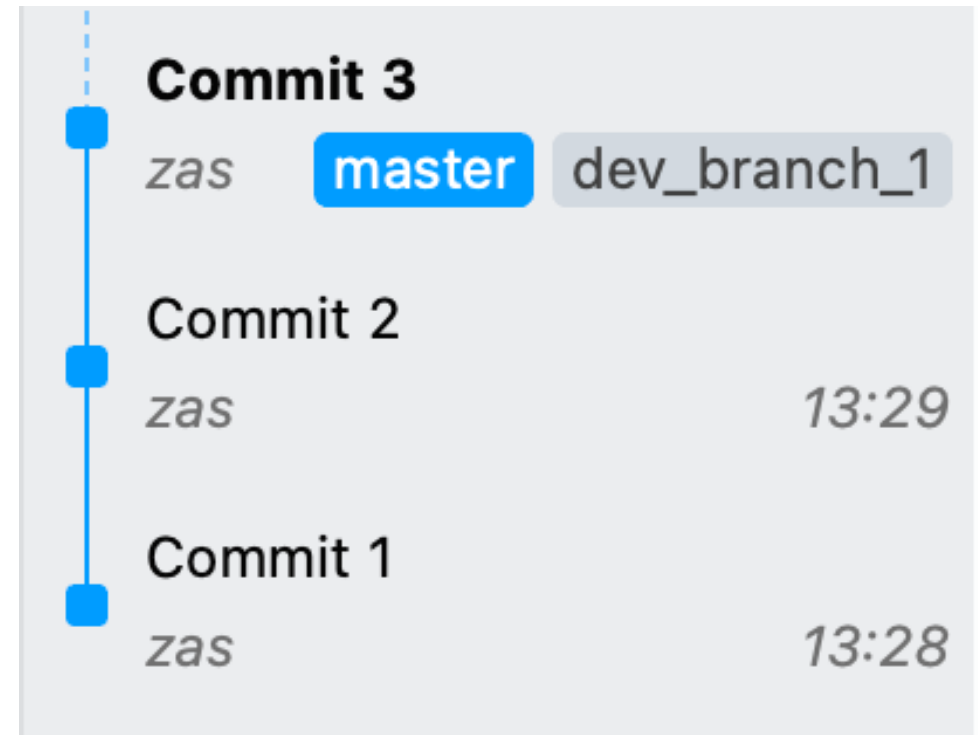
# Branch und Merge

**Create** branch dev\_branch\_1



Erstelle branch dev\_branch\_1

```
$ git branch dev_branch_1
```



# Branch und Merge

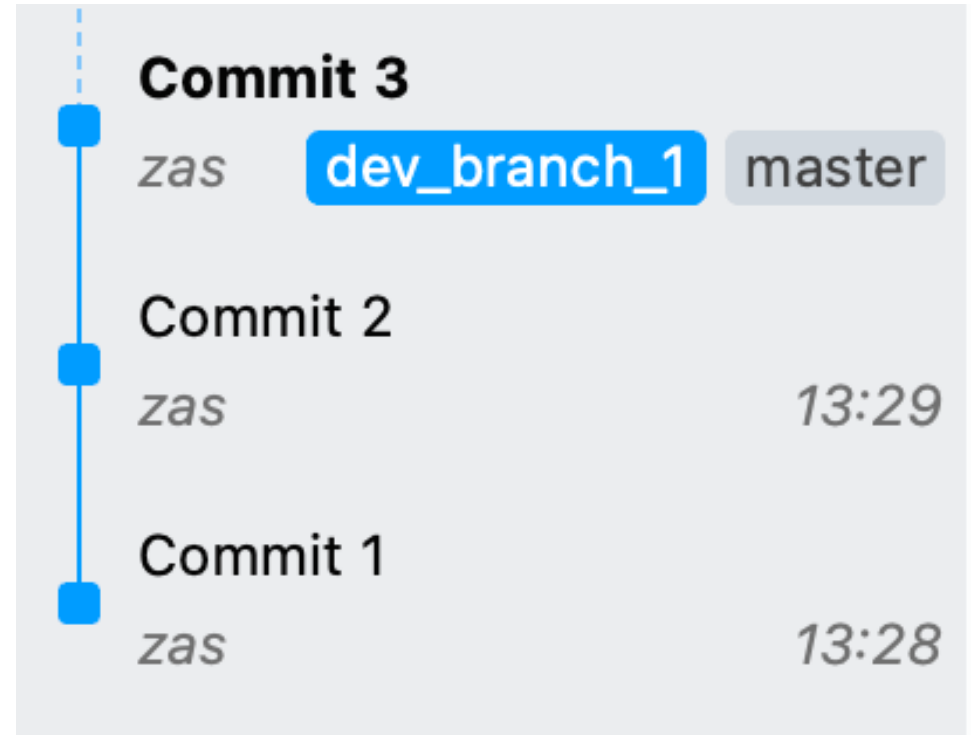
**Checkout** branch dev\_branch\_1

Prüfen Sie, in welchem Zweig wir uns befinden

```
$ git branch
```

Checkout branch dev\_branch\_1

```
$ git checkout dev_branch_1
```



# Branch and Merge

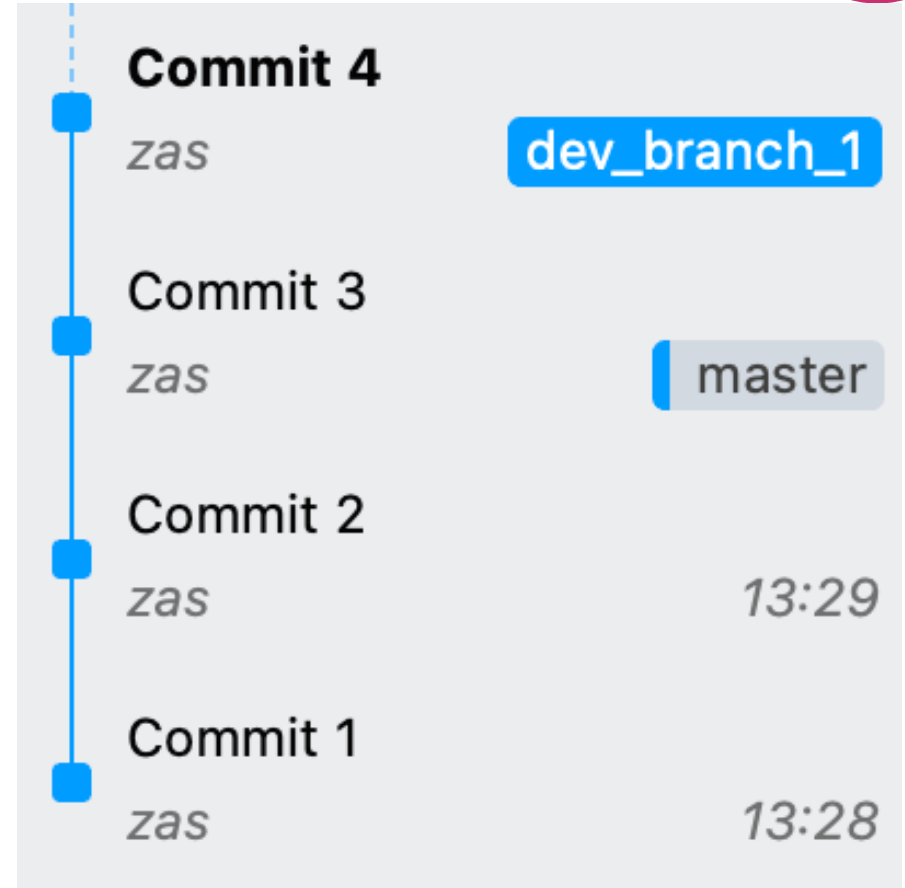
**Commit** on dev\_branch\_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 4"
```



# Branch und Merge

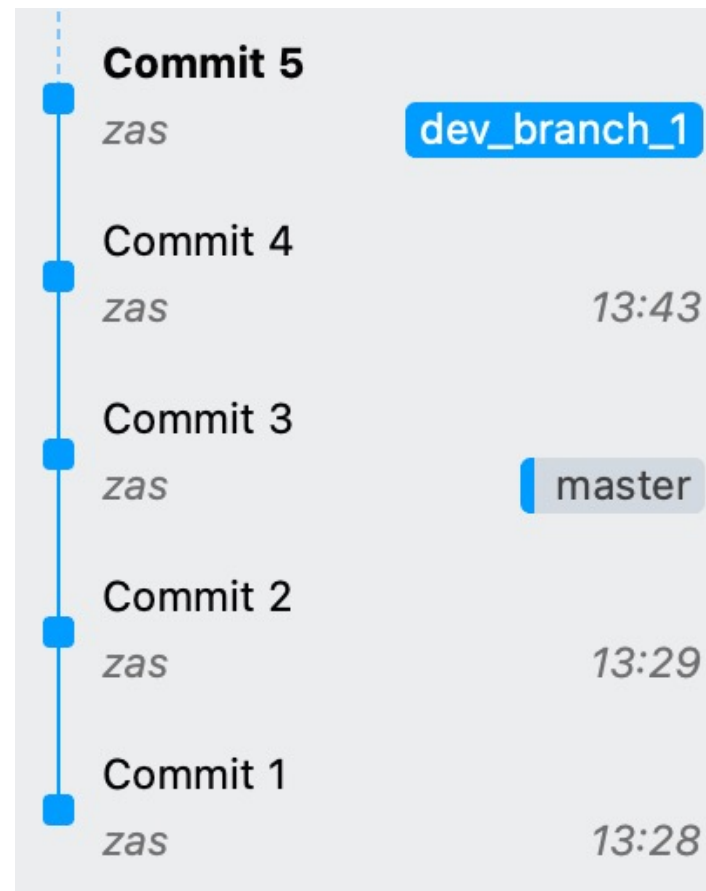
**Commit** on dev\_branch\_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 5"
```



# Branch und Merge

**Commit** on master branch

Checkout master branch

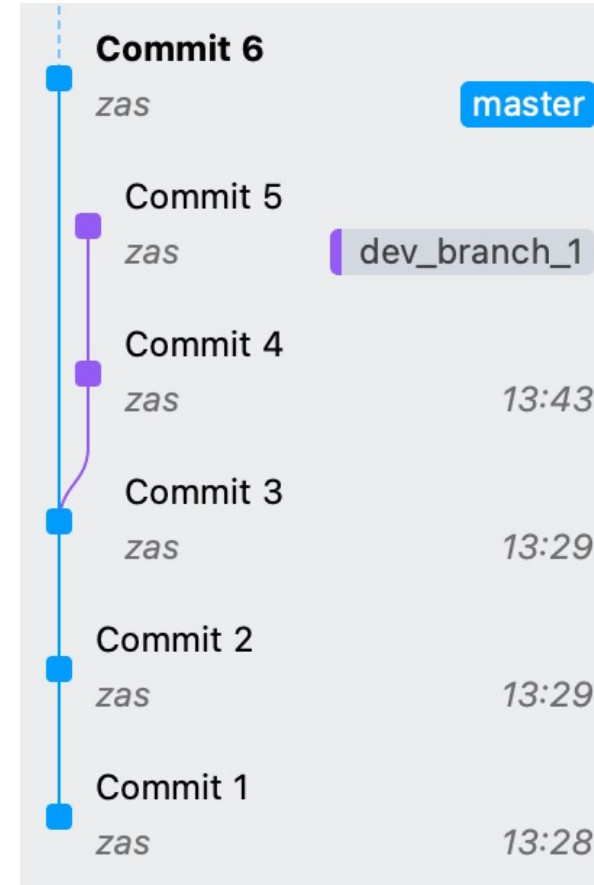
```
$ git checkout master
```

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 6"
```





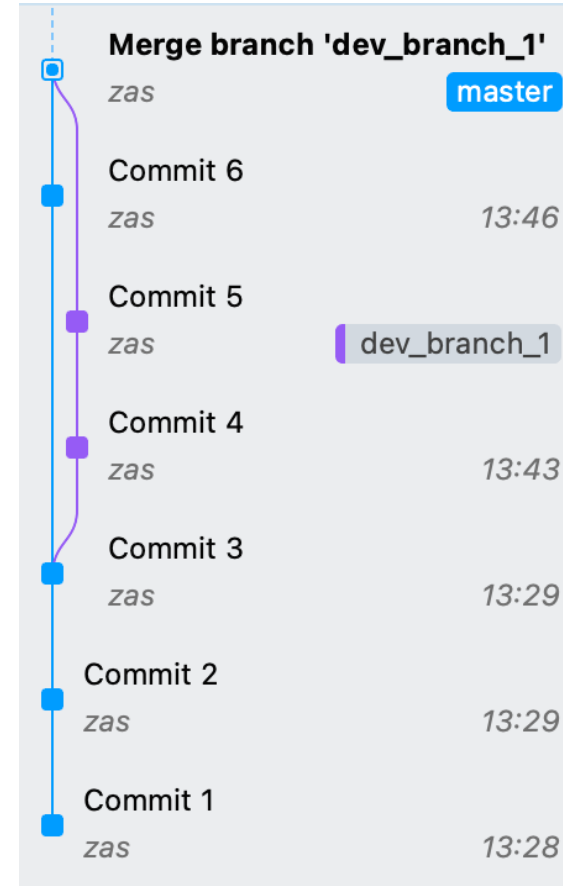
# Branch und Merge

## Three way merge master and dev\_branch\_1

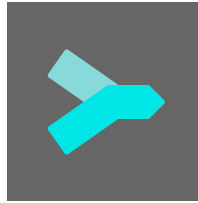


Merge dev\_branch\_1 into master

```
$ git merge dev_branch_1
```



# Demonstration



~/.work/repo/edu/car/car-course

Locations: did-course, car-course, car-exams, znotes, syd-course

Commits: 1 unstaged file, master, origin/master

Files: 00-solution.tex, 03-controlunit.tex, 04-simulation.tex, 05-deployment.tex, 05-deployment\_ebs3.tex, 08-CAR-Labor-SCR-d.pdf, 08-CAR-Labor-SCR-f.pdf, CAR-Labor-SCR-s.pdf, CAR-Labor-SCR-t.pdf

Summary: 16 files changed: -15, +110

Subsection: Simulation:

```
done:
beq x2, x2, main # infinite loop
end(minted)
```

Each instruction takes one clock cycle => 19 are executed (addi x5, x0, 0 not executed because of previous beq; addi x2, x0, 1 not executed because of jal).

On the EBS2 board @ 66MHz  $\rightarrow \frac{19}{66\text{M}} = 287.9 \text{ ns}$ .

On the EBS3 board @ 50MHz  $\rightarrow \frac{19}{50\text{M}} = 380 \text{ ns}$ .

Subsection: Umsetzung:

Le `\texttt{mainDecoder}` peut être écrit en VHDL. Pour cela, vous pouvez analyser l'exemple de code `\ref{fig:riscv-mainDecoder-code}` ci-dessous et l'adapter en conséquence.

Dans HDL Designer, lorsque vous sélectionnez le type de contenu d'un bloc, choisissez `\texttt{VHDL File -> Architecture}`, et contrôlez que le langage est défini sur `\texttt{VHDL 2008}`. Sur la page suivante, `\texttt{Architecture}` correspond au nom de la vue (un bloc peut avoir différents contenus) et `\texttt{Entity}` au nom du bloc (`mainDecoder` par exemple).

63 `\begin{center}`

64 `\centerline{\includegraphics[width=0.9\paperwidth]{scr/sol/simulation.pdf}}`

65 `\end{center}`

66 `\begin{center}`

67 `\centerline{\includegraphics[width=0.9\paperwidth]{scr/sol/simulation.pdf}}`

68 `\end{center}`

69 `\begin{center}`

70 `\centerline{\includegraphics[width=0.9\paperwidth]{scr/sol/simulation.pdf}}`

71 `\end{center}`

72 `\begin{center}`

73 `\centerline{\includegraphics[width=0.9\paperwidth]{scr/sol/simulation.pdf}}`

74 `\end{center}`

75 `\end{center}`

figriscv-mainDecoder-code:

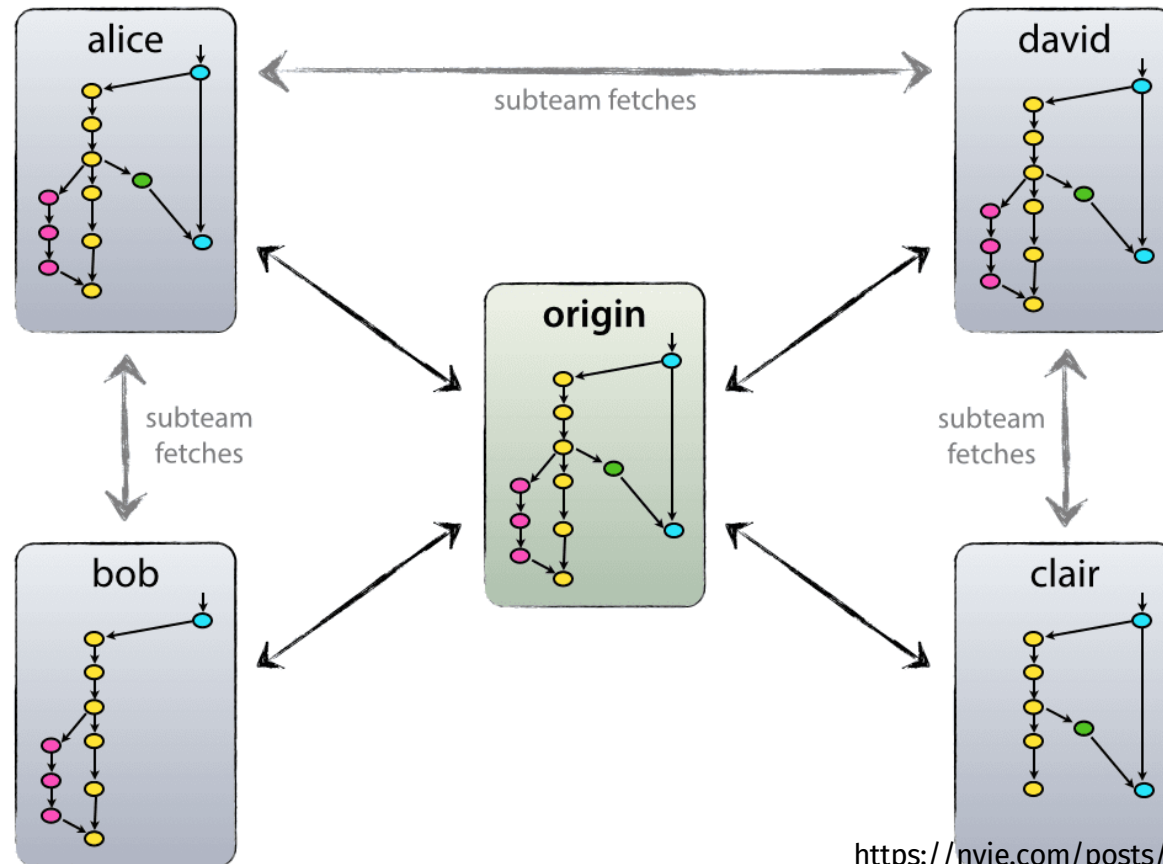
```
\opt{f}{\captionof{figure}{Exemple de code MainDecoder}}
\opt{d}{\captionof{figure}{MainDecoder Code-Beispiel}}
\label{fig:riscv-mainDecoder-code}

\subsection{ALU}
\opt{f}{
  L'ALU réalise les fonctions arithmétiques et logiques selon la table suivante:
}
\opt{d}{
  Die ALU realisiert die arithmetischen und logischen Funktionen gemäß der folgenden Tabelle:
}
\begin{table}[h]
\caption{ALU Functions}
```



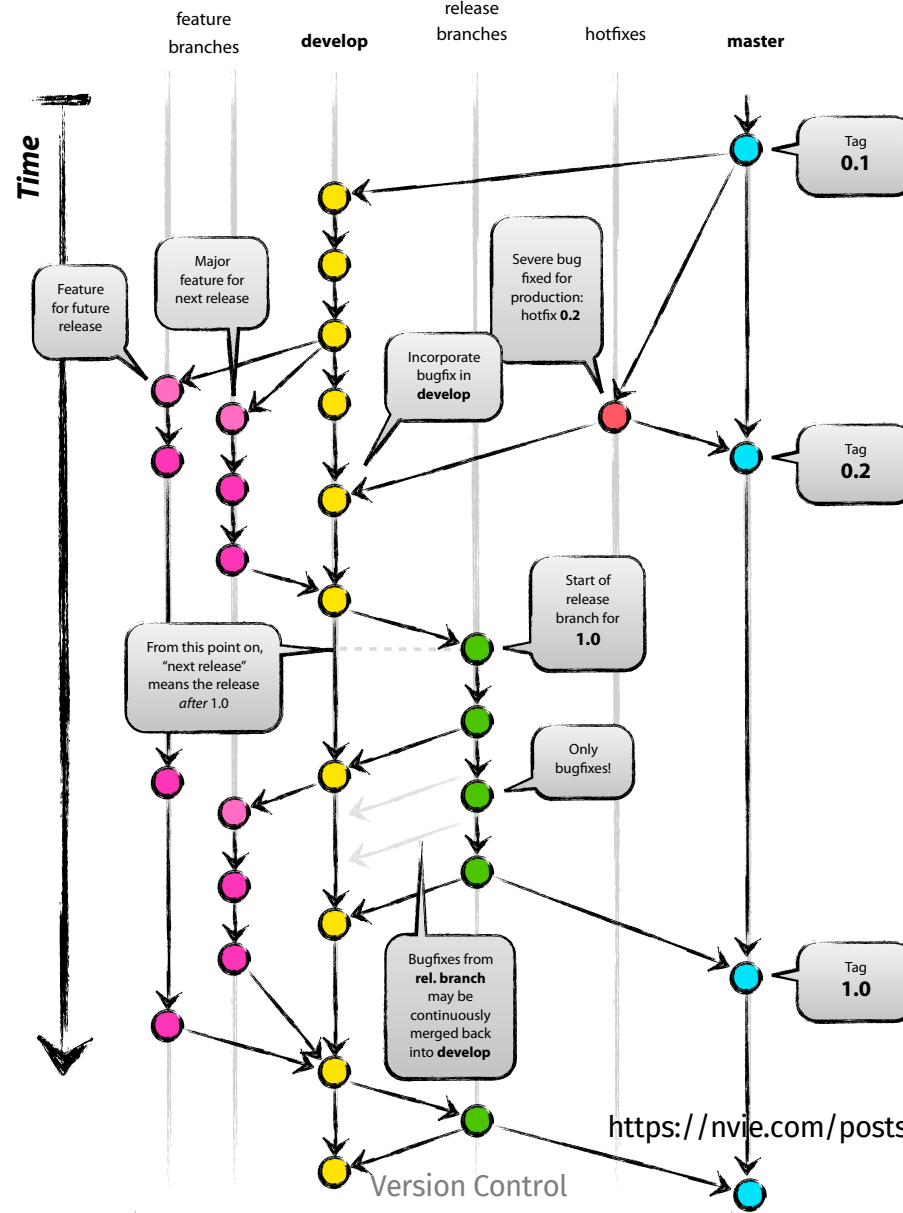
Gitflow

# Gitflow Zusammenarbeit

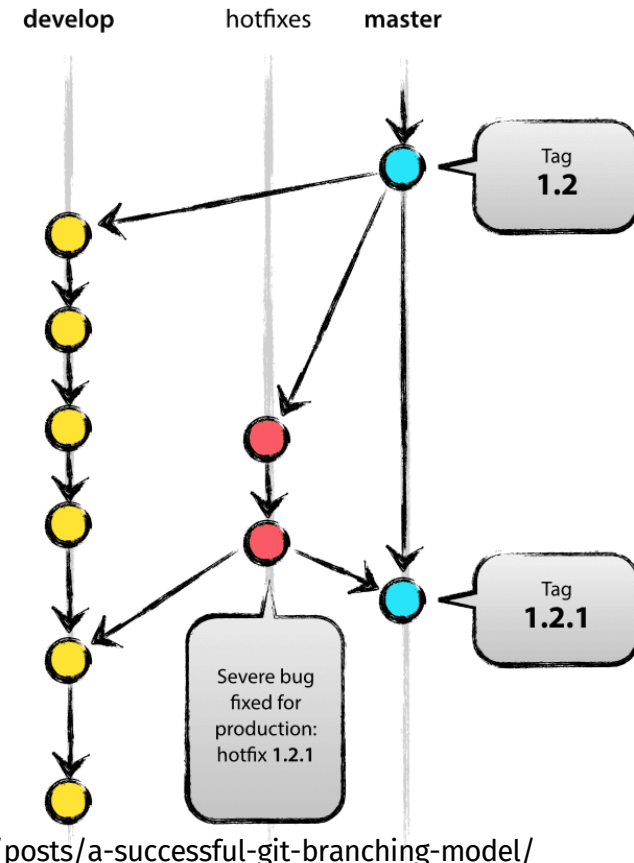
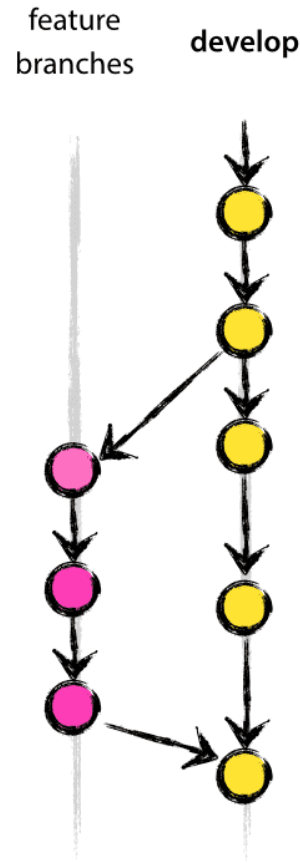
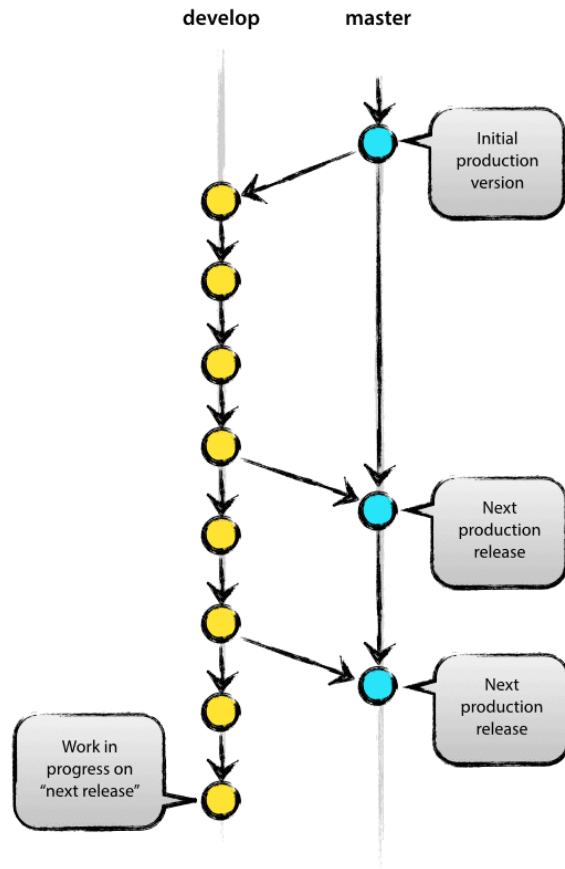


<https://nvie.com/posts/a-successful-git-branching-model/>

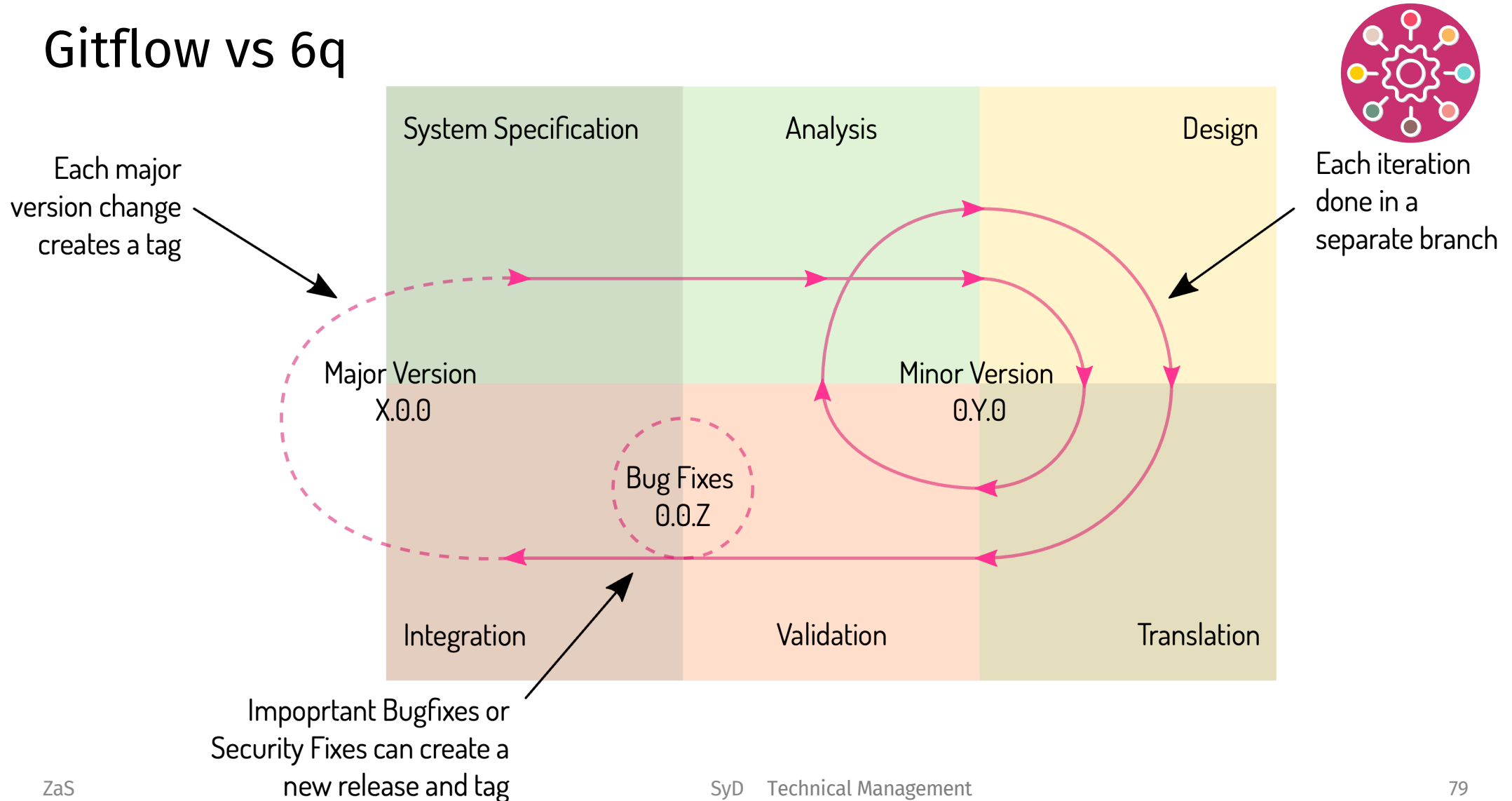
# Gitflow



# Gitflow Branching



# Gitflow vs 6q





Git CI/CD

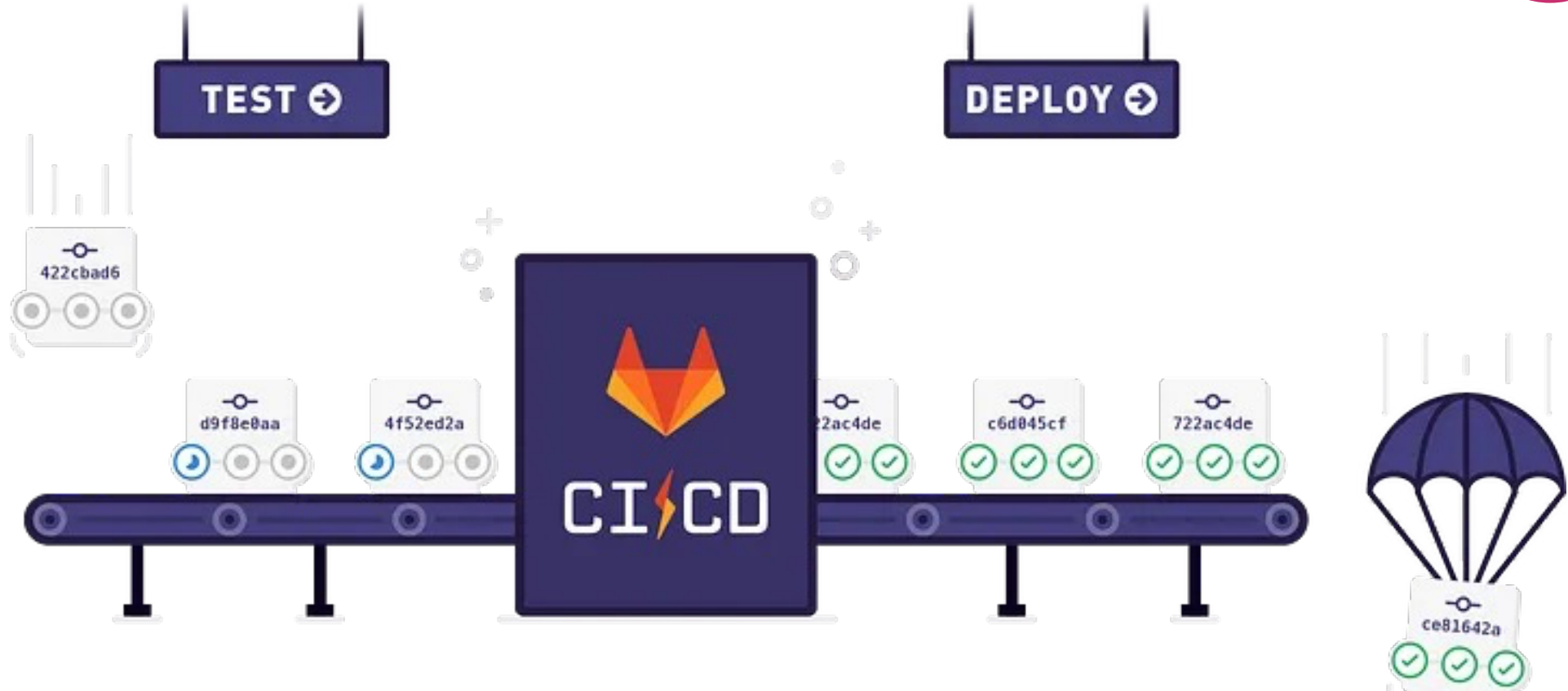


# Was ist CI/CD?



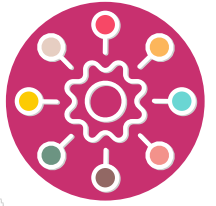
- Bei der kontinuierlichen Integration (Continuous Integration, CI) werden Codeänderungen häufig in ein gemeinsames Repository integriert, das dann automatisch erstellt und getestet wird.
- Continuous Delivery (CD) geht noch einen Schritt weiter, indem Codeänderungen automatisch in produktionsähnlichen Umgebungen für weitere Tests und Validierungen bereitgestellt werden.
- Automatisierte Tests sind eine wichtige Komponente von CI/CD, da sie dazu beitragen, Bugs und andere Probleme frühzeitig im Entwicklungsprozess zu erkennen.
- Beliebte Tools sind GitLab CI/CD, Github Actions, Jenkins, CircleCI und Travis CI.

# Was ist CI/CD?

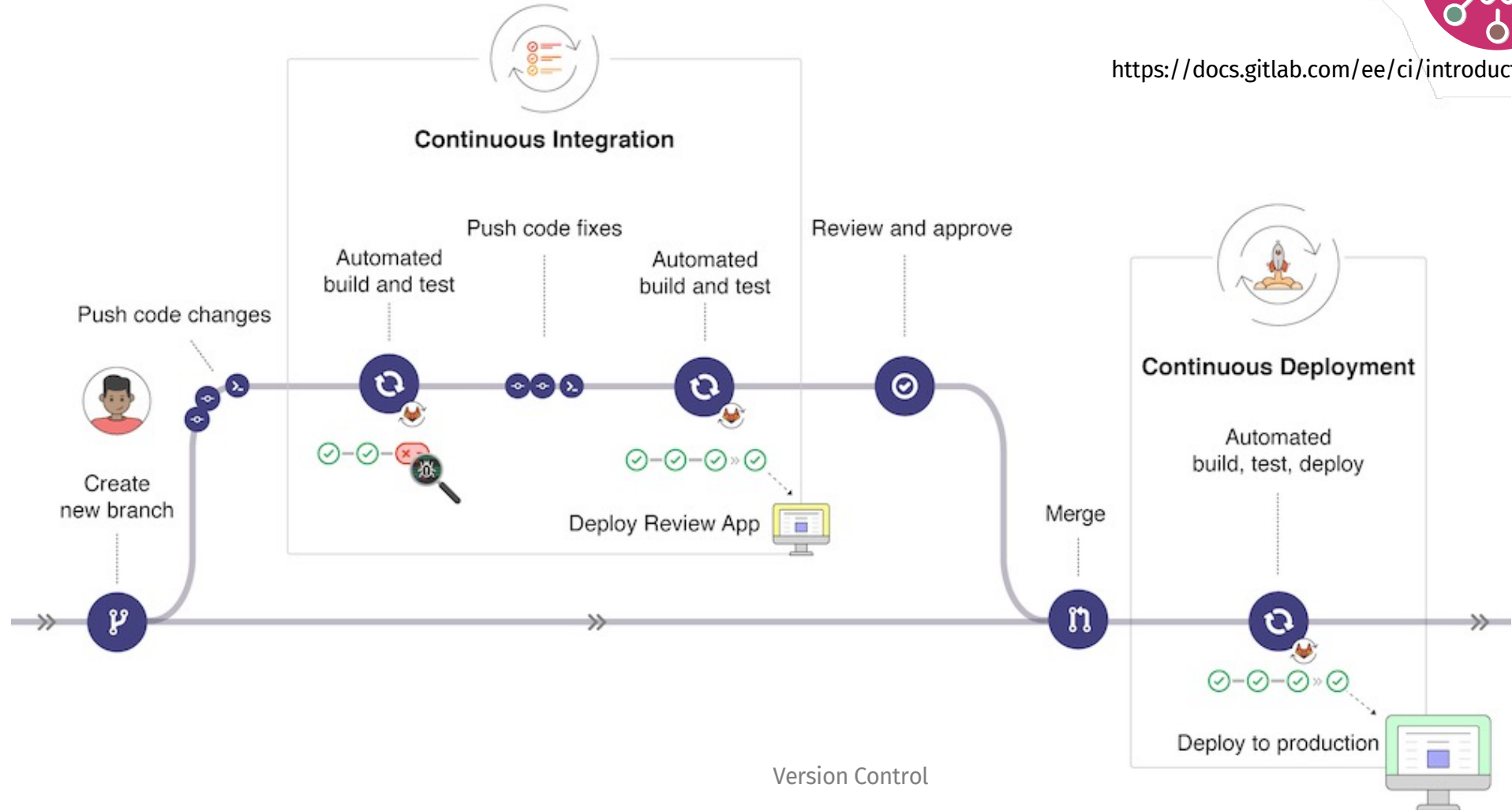


<https://about.gitlab.com/features/continuous-integration/>

# Gitlab Workflow



<https://docs.gitlab.com/ee/ci/introduction/>

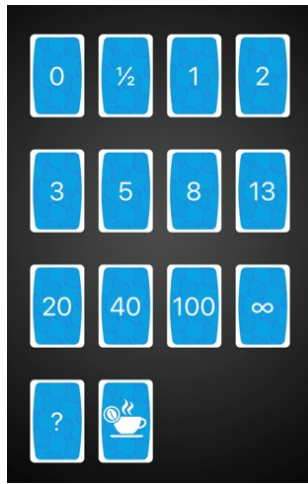


# Abschätzung von Aufgaben

## Abschätzung 4

Erstelle einen Star Wars  
Star Destroyer (4784pcs)

X Punkt(e)

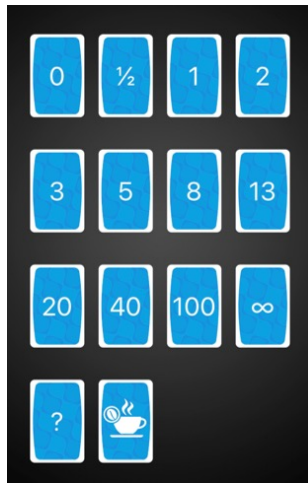


# Abschätzung von Aufgaben

## Abschätzung 5

Erstelle einen Star Wars  
Star Destroyer (37pcs)

X Punkt(e)





SYSTEMS DESIGN / INTRODUCTION TO GIT - PART A



## Introduction to git - Part A

Installation & Setup



### Contents

|                                |   |
|--------------------------------|---|
| 1 Goal .....                   | 1 |
| 2 Installation .....           | 2 |
| 3 Markdown .....               | 5 |
| 4 Outro .....                  | 7 |
| A GIT commands .....           | 8 |
| B Most used Git commands ..... | 9 |

SYSTEMS DESIGN / INTRODUCTION TO GIT - PART B



## Introduction to git - Part B



### Contents

|                           |    |
|---------------------------|----|
| 1 Goals .....             | 2  |
| 2 Outils .....            | 2  |
| 3 Basis Operationen ..... | 3  |
| 4 Branch and Merge .....  | 11 |
| 5 Gitgraph .....          | 15 |
| 6 Gitflow .....           | 16 |
| 7 Extras .....            | 18 |